

JAVA PROJECT

CYBER CRIME INVESTIGATION SYSTEM

A Java-Based Case, Suspect and Evidence Management System

Submitted By:

Deepti Singh

25BAI10058

Program: B.Tech – CSE (AI & ML)

Semester: III

1. ABSTRACT

The Cyber Crime Investigation System is a Java-based project designed to make the management of cyber crime investigation records easier and more organized. The system allows the user to maintain details about different cases, suspects, and evidence related to an investigation.

The project is developed using Java and makes use of important concepts such as classes and objects, ArrayList, file handling, exception handling, and Java Swing. A simple graphical interface has also been created so that the user can perform common operations without depending completely on the command-line interface.

The system currently supports adding, viewing, and searching cases along with adding and viewing suspect and evidence information. Case details are stored using file handling, which allows the application to load previously stored cases when it starts.

2. INTRODUCTION

With the increasing use of computers, smartphones, and the internet, cyber crimes have also become an important area of investigation. During an investigation, a large amount of information such as case details, suspect information, and evidence needs to be properly maintained.

This project was created as a simple way to organize such information using Java. Instead of keeping all the information in an unorganized form, the system separates cases, suspects, and evidence into different classes and manager classes.

The project also includes a basic GUI that makes the system easier to use and gives a more practical feel compared to a completely console-based program.

3. PROBLEM STATEMENT

While working with investigation records manually, it can become difficult to keep track of different cases and their related information.

The main idea behind this project is to create a simple system where a user can:

- Add new cyber crime cases
- View available cases
- Search for a particular case
- Add suspect details
- View suspect records
- Add evidence details
- View evidence records
- Store case information using files

4. OBJECTIVES

The main objectives of this project are:

- To understand how Java can be used for a practical application.
- To organize cyber crime case information in a structured way.
- To maintain basic suspect and evidence records.
- To implement file handling for storing case information.
- To provide a simple search facility for cases.
- To create a user-friendly GUI using Java Swing.
- To apply Object-Oriented Programming concepts in a real project.

5. TECHNOLOGIES USED

- I. Technology| Use in the Project
- II. Java| Development of the complete application
- III. Java Swing| Creating the graphical user interface
- IV. ArrayList| Temporarily managing records
- V. File Handling| Saving and loading case information
- VI. VS Code| Writing and running the Java code

6. MAIN FEATURES

Case Management

The case module is used to manage basic information about cyber crime cases. A case contains its Case ID, crime type, description, severity, and current status.

The user can add a new case, view existing cases, and search for a case using its Case ID.

Suspect Management

The suspect module stores basic details of people associated with an investigation. It includes the Suspect ID, name, contact number, and address.

Evidence Management

The evidence module is used to record information about evidence related to an investigation. It stores an Evidence ID, type of evidence, and its description.

Graphical User Interface

A Java Swing interface has been added to make the project easier to operate. The main window provides separate buttons for case, suspect, and evidence-related operations.

7. PROJECT STRUCTURE

The project is divided into different classes so that each class has a specific purpose.

"Case.java"

This class represents an individual cyber crime case. It stores details such as Case ID, crime type, description, severity, and status.

"CaseManager.java"

This class manages case-related operations. It is responsible for adding, viewing, searching, and loading cases.

"Suspect.java"

This class represents a suspect and contains basic suspect information.

"SuspectManager.java"

This class is used to add and display suspect records.

"Evidence.java"

This class represents evidence associated with an investigation.

"EvidenceManager.java"

This class handles adding and displaying evidence records.

"FileManager.java"

This class is used for file-related operations, mainly for storing case information.

"Main.java"

This class contains the original console-based menu through which the different operations can be performed.

"CaseManagementGUI.java"

This is the GUI part of the project. It provides buttons and input forms for performing the major operations of the system.

8. BASIC WORKING OF THE SYSTEM

The system starts by creating the required manager objects. When the application starts, previously stored case information is loaded from the data file.

The user can then select an operation from the GUI.

For example, while adding a case, the user enters the Case ID, crime type, description, and severity. The system creates a Case object and passes it to the CaseManager. The case is then added to the list and stored using the file-handling mechanism.

Similarly, the user can enter details of a suspect or evidence through the GUI.

For searching, the user enters a Case ID and the system checks the available cases to find the matching record.

9. FILE HANDLING

File handling is used so that case information does not have to be entered again every time the application is started.

The case records are stored in:

`"data/cases.txt"`

When a new case is added, its information is written to the file. When the program starts again, the "CaseManager" reads the file and loads the stored cases.

This was useful for understanding how Java can work with external files instead of keeping all information only inside the program.

10. OOP CONCEPTS USED

Classes and Objects

The project uses separate classes such as Case, Suspect, and Evidence. Objects of these classes represent individual records.

Encapsulation

The information and operations related to a particular type of record are kept within the appropriate classes.

Abstraction

The manager classes provide simple methods such as "addCase()" and "searchCase()" so that the main program does not need to handle every internal operation itself.

Modularity

The project is divided into multiple classes instead of putting the complete code in one file. This makes the code easier to understand and modify.

11. SAMPLE RECORDS

Case 1

Case ID: C001

Crime Type: Phishing

Description: Fake banking website used to collect user credentials.

Severity: High

Status: Open

Case 2

Case ID: C002

Crime Type: Ransomware Attack

Description: A college computer system was affected by malware that encrypted important files and restricted access.

Severity: High

Status: Open

Suspect

Suspect ID: 102

Name: Arjun Verma

Contact: 9123456780

Address: Indore, Madhya Pradesh

Evidence

Evidence ID: 202

Evidence Type: System Log

Description: Server logs showing suspicious login activity shortly before the ransomware incident.

13. ADVANTAGES

- The system is simple and easy to understand.
- Cases can be searched using their Case ID.
- Different types of information are separated into different classes.
- Case information can be stored using file handling.
- The GUI makes the application easier to operate.
- The project demonstrates practical use of Java and OOP concepts.
- The modular structure makes future changes easier.

14. LIMITATIONS

Since this is a basic academic version, there are a few limitations.

- The current system uses file-based storage instead of a database.
 - Suspect and evidence records are maintained during the current program session.
 - There is no login or authentication system.
 - Actual evidence files cannot be uploaded.
 - The GUI is kept simple and can be improved further.
- These limitations can be addressed in future versions of the project.

15. FUTURE SCOPE

There are several ways in which this project can be improved in the future.

A MySQL database can be added to provide proper permanent storage for cases, suspects, and evidence. A login system can also be introduced so that only authorized users can access investigation records.

Other possible improvements include uploading evidence files, advanced case searching, filtering records by severity or status, generating investigation reports, and improving the overall GUI design.

16. CONCLUSION

The Cyber Crime Investigation System helped us understand how different Java concepts can be combined to build a practical application.

Through this project, we implemented separate classes for cases, suspects, and evidence and used manager classes to handle their operations. File handling was used to store case information, while Java Swing was used to create a simple graphical interface.

The project provides a basic but functional foundation for managing cyber crime investigation records. It can be further developed by adding database connectivity, authentication, better GUI design, and more advanced investigation features.

Overall, this project gave practical experience in Java programming, Object-Oriented Programming, file handling, and GUI development.

17. REFERENCES

1. Java programming concepts and class documentation
2. Java Swing documentation
3. Java File Handling concepts
4. Object-Oriented Programming concepts in Java

THANK YOU!!